

Towards Hybrid Link Traversal: Challenges and Research Directions for Heterogeneous Dataspaces

Ruben Eschauzier¹, Ruben Taelman¹

¹Department of Electronics and Information Systems, Ghent University – imec

Abstract

Decentralized dataspace preserve data sovereignty by keeping data at its source, but querying a large number (thousands) of autonomous sources with heterogeneous interfaces introduces significant challenges. Traditional Federated Query Processing (FQP) engines fail to scale to this size, while existing Link Traversal-based Query Processing (LTQP) systems can only handle Linked Data documents. By ignoring the (query) capabilities of alternative data interfaces, current engines fail to take advantage of the performance gains offered by more expressive interfaces. To address this, we advocate for Hybrid FQP-LTQP, an execution strategy that combines the dynamic runtime discovery of link traversal with the performance benefits of delegating complex sub-queries to capable server-side interfaces. This paper reviews the state-of-the-art in hybrid traversal and identifies the critical challenges hindering its implementation: establishing exclusive groups without prior knowledge, integrating dynamically identified sub-queries into query plans, enabling reliable data access interface discovery, and deduplicating results obtained from these interfaces. Solving the challenges identified in this paper is a strict prerequisite for making hybrid decentralized querying practically viable across heterogeneous dataspace. Consequently, future research must prioritize these open problems to engineer the next generation of scalable, hybrid FQP-LTQP engines.

Keywords

Link Traversal-based Query Processing, Hybrid Link Traversal, SPARQL, Dataspace, Query Optimization

This paper addresses (Issue #2) of the W3C Dataspace Community Group.

1. Introduction

While centralizing data into warehouses or lakes can benefit query engine performance, it is antithetical to the core principles of *dataspace*. Dataspace promote a decentralized ecosystem where data remains at the source, managed by independent participants under agreed-upon governance models. This architectural shift introduces significant technical challenges for query engines, which must now discover and query data across a vast network of sources, each enforcing its own usage policies.

Centralized querying requires collecting large volumes of proprietary or sensitive data in a single source, which conflicts with the requirements for fine-grained sovereignty and minimized data replication. Conversely, traditional *federated* query processing approaches (FQP) [1] support decentralization but typically assume a static federation of a small number (10–100) of uniform, expressive endpoints (e.g., SPARQL endpoints) [2]. Closely related is heterogeneous FQP [3, 4], which operates similarly but allows different source interfaces, such as Triple Pattern Fragments (TPF) [5], and WiseKG [6]. Enforcing complex usage policies [7] on such heavyweight interfaces significantly impacts performance [8], and these engines struggle to scale to the thousands of sources characteristic of a dataspace.

In contrast, realistic dataspace environments are characterized by a massive scale of permissioned sources and high interface diversity [9]. Beyond the structured fragments managed by traditional heterogeneous federated engines, dataspace participants may, for example, expose data through generic HTTP documents or derived views [10]. Because these participants are autonomous and decentralized, their availability and specific query capabilities are often unknown until the point of execution. Addressing this requires a *hybrid* federation approach: one that can dynamically discover and interoperate

The Third International Workshop on Semantics in Dataspace, co-located with the Extended Semantic Web Conference, June 01, 2025, Portorož, Slovenia

✉ ruben.eschauzier@ugent.be (R. Eschauzier); ruben.taelman@ugent.be (R. Taelman)

ORCID 0000-0002-6475-806X (R. Eschauzier); 0000-0001-5118-256X (R. Taelman)



© 2025 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

with highly heterogeneous sources at runtime, without the overhead of prior centralization or static configuration.

Link Traversal-based Query Processing (LTQP) is an approach that meets these needs by discovering data sources during execution via hypermedia links found in earlier results. Starting from seed references (e.g., a self-description or catalog entry), it follows links asynchronously. While LTQP is traditionally defined as an approach for querying over federated Linked Data documents by following links at runtime, data within a dataspace is often exposed through diverse access interfaces that may not support standard document-based traversal and querying.

We advocate for *Hybrid FQP-LTQP*, which we define as a query execution paradigm that advances beyond traditional Linked Data Document-centric traversal. It enables query engines to traverse, integrate, and actively leverage the computational capabilities of heterogeneous, expressive data interfaces alongside standard Linked Data Documents. Possible examples of such expressive interfaces are SPARQL endpoints and TPF [5] (Figure 1).

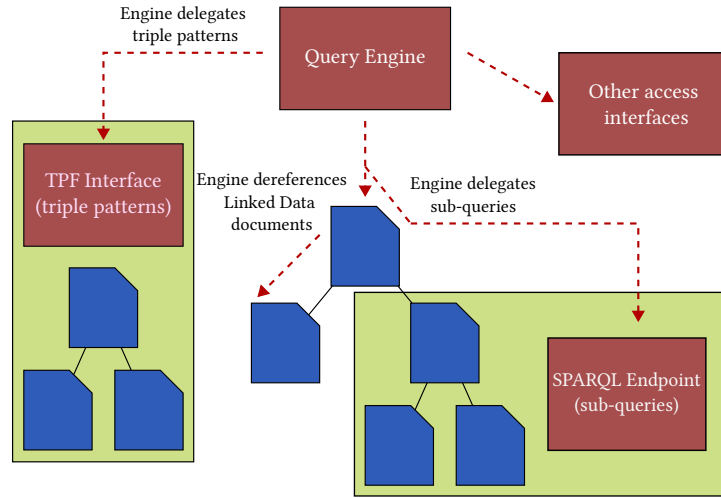


Figure 1: Hybrid FQP-LTQP leveraging the varying levels of expressiveness of heterogeneous sources during the traversal query execution

To leverage these varied levels of expressivity, a hybrid engine must offload computation to the server whenever possible. For instance, a SPARQL endpoint can execute complex sub-queries locally, while document-oriented interfaces cannot. By leveraging internal indexing and local data knowledge, the endpoint could produce results far more efficiently than a client-side LTQP engine. Offloading these tasks to capable endpoints could significantly reduce overall query latency. However, building such a system introduces several unresolved challenges. In this paper, we review the state-of-the-art in hybrid querying, analyze related work, and outline open issues for future research.

2. Related Work

While Hybrid FQP-LTQP has previously been explored in literature, its exact definition varies across studies. Early works [11, 12, 13] typically define a “hybrid” approach as the combination of live link traversal with traditional centralized or federated querying mechanisms operating separately. In contrast, our work employs a strictly live-traversal process, redefining the “hybrid” aspect as the traversal over heterogeneous data sources within a single process.

2.1. Hybrid Traversal - Local vs Remote

A branch of FQP-LTQP uses precomputed indexes, either located within the decentralized environment or computed locally, to speed up query execution or source discovery. The first of such approaches

uses precomputed indexes to rerank sources based on relevance to the query’s triple patterns and joins, retrieving them to execute queries [14].

Another approach is to store entire RDF graphs in local stores to quickly execute parts of the query. As a result, non-blocking index-based operators [15] can be used to leverage the locally computed indexes for faster query execution.

A major limitation is the freshness of the data in the store and indexes, which can quickly go stale for high-velocity data sources. In addition, these preliminary works deviate from our definition of hybrid FQP-LTQP, as they rely solely on Linked Data Document-centric traversal to populate local caches or use decentralized indexes only to optimize the Linked Data Document-based traversal process.

2.2. Hybrid Traversal - Multiple Processes

The methods in the literature falling within our definition of hybrid FQP-LTQP use two separate query processes to obtain data from heterogeneous sources. First, coherence-aware query processing uses a centralized store to obtain initial query results quickly. Using coherence estimation, it determines which data from this store is likely stale. The stale data is then discovered and queried using the traversal-based second process during query execution. Finally, the results are combined to provide the query answer.

A second approach is a hybrid method that executes a federated query over known SPARQL endpoints concurrently with traversal-based query processing over URIs [12]. While this mitigates the issue of data freshness by querying sources at runtime rather than relying on static indexes, it requires the relevant SPARQL endpoints to be configured prior to query execution. Consequently, the system cannot utilize new SPARQL endpoints discovered during the traversal process.

2.3. Hybrid Traversal - Closed Systems

Other approaches integrate into the dataspace itself. ESPRESSO [16] is such an approach; it is a framework around the Solid dataspace specifically that defines several apps in the dataspace to perform certain (optimization) tasks. In the Solid environment, data is stored in personal data vaults. These vaults use the Linked Data Platform [17] to arrange the data into a folder-like structure. To avoid traversing the entire pod to query it, ESPRESSO uses the *Brewmaster* application to locally create indexes of the pod and uses the *CoffeeFilter* application to search its contents. In addition, ESPRESSO uses an overlay network to distribute end-user queries to relevant data resources across Solid servers, where each Solid server is mapped to a federated database node in the overlay network.

Unlike approaches based on link traversal or service discovery within Solid, ESPRESSO relies on an explicitly configured overlay network, with no mechanism for dynamically discovering participating servers or query endpoints. Furthermore, currently ESPRESSO only supports distributed keyword-based search, it does not support SPARQL queries over the sources.

While these approaches do hybrid link traversal, they significantly deviate from our interpretation of hybrid FQP-LTQP. This is in contrast to the setting described in this paper where the discovered sources themselves are heterogeneous, serendipitously discovered, and cannot be assumed to be present for every source within the dataspace.

2.4. Hybrid Link Traversal over Heterogeneous Sources

State-of-the-art LTQP engines, most notably Comunica Link Traversal [18, 19], demonstrate hybrid capabilities by dynamically federating over discovered SPARQL endpoints, TPF interfaces, and hyper-media documents. However, these systems are currently limited by a triple pattern level-only strategy: they fail to leverage the full expressivity of sophisticated interfaces. For instance, upon discovering a SPARQL endpoint, existing engines restrict interaction to simple *triple pattern* lookups and do not attempt sending sub-queries.

This often results in the client retrieving high volumes of intermediate results for performing local joins, reducing the optimization and latency benefits of more expressive server-side execution.

3. Challenges with Hybrid Link Traversal

3.1. Establishing Exclusive Groups

In FQP, *exclusive groups* are sets of triple patterns within a query that can *only* be answered by a single source [20]. Instead of retrieving results for individual triple patterns and joining them locally, the engine can delegate the entire group as a single sub-query (e.g., a Basic Graph Pattern) to the remote endpoint. This technique significantly reduces the number of remote requests and processed intermediate results [21, 20].

If a query engine cannot verify that a group of patterns belongs *exclusively* to a single source, it cannot safely dispatch them as a conjunctive sub-query (BGP). Doing so risks *incomplete results* because the endpoint will execute a server-side join. If the endpoint lacks data for one of the patterns, or if valid join partners reside on a different server, the server-side join execution will filter out intermediary results that could have been successfully joined with data from another source.

FQP engines like *FedX* [20], *HiBISCuS* [22], *Comunica* [18], and *FedUp* [21] rely on either prior knowledge on the content of the federation members, or issue queries (such as ASK) to determine these groups.

To delegate joins to SPARQL endpoints, Hybrid FQP-LTQP engines must identify exclusive groups. Traditional FQP algorithms require all sources to be known during optimization; this requirement is incompatible with the unbounded nature of LTQP [23]. Even within a bounded web, requiring engines to dereference all data prior to execution eliminates the streaming capabilities of current LTQP systems [18, 24]. Therefore, Hybrid FQP-LTQP demands novel approaches for exclusive group identification.

3.1.1. Research Avenues

Determining exclusive groups in hybrid link traversal requires comprehensive knowledge of source coverage, specifically information about what data is *only* available at a single source. Several research avenues exist to obtain this information.

Discoverable Indexes: Indexes can be effectively used to catalogue the data sources present in a dataspace. These indexes could be deployed as services, similar to approaches like ESPRESSO [16], pointing towards source URIs or summary statistics. Using indexes to quickly determine relevant sources, LTQP engines can apply FQP techniques to establish exclusive groups and perform source selection. This would move the problem from traversal to federation, with new challenges such as index completeness and maintenance.

Caching: Similar to discoverable indexes, caches provide prior knowledge about the data sources present in a dataspace. The query engine can perform a ‘simulated traversal’ offline over the cached data to identify relevant sources. Any gaps in this knowledge can then be supplemented by live traversal prior to or during execution

The challenge is determining when a cache is “complete” for the current query, as missing or stale data can affect the correctness of identified exclusive groups. For example, if the cached data originates from a different query, the cached traversal can differ due to URI reachability [23] depending on query content.

Authoritativeness Assumptions: Establishing exclusive groups in a decentralized dataspace could be achieved through the notion of *source authority* [25]. Under standard Link Traversal, any reachable source can assert triples about any subject (e.g., a third-party source asserting $\langle P1 \rangle \text{ foaf:knows } \langle P2 \rangle$), creating a “wild west” of data where no single source can be deemed the exclusive authority for a topic. This lack of authority prevents the query engine from determining if a set of patterns can be safely delegated to a single dataspace or endpoint. Previous work on guided link traversal [26]

demonstrates that introducing structural assumptions enables further optimizations. Applying this principle, a potential solution restricts the scope of validity, assuming a data source acts as the *sole authority* for triples where the subject corresponds to the source’s own URI (or a URI within its controlled namespace). By enforcing this *Subject Authority Constraint*, the query engine can statically determine that all triples matching $\langle \text{SourceA}/\text{posts}/\text{post1} \rangle \text{ ?p ?o}$ reside exclusively at *Source A*¹. This allows the engine to safely treat such patterns as an exclusive group, enabling the delegation of complex sub-queries (e.g., BGPs) to hybrid sources like SPARQL endpoints without the risk of incomplete results.

3.2. Integrating Dynamically Delegated Sub-queries into the Query Plan

In the traditional optimize-then-execute approach, used by Comunica-link-traversal [18, 19] and SQUIN [24] (except in [27]), the query plan is constructed before execution and evaluated over dynamically discovered sources.

However, exclusive groups are only identified during query execution, when expressive interfaces such as SPARQL endpoints are discovered through link traversal.

Because the execution plan is already fixed, integrating an exclusive group from a newly discovered interface requires dynamically transforming the plan to accommodate the subquery.

Example 3.1. As an example, consider a query plan with the join structure $(T_1 \bowtie T_2) \bowtie (T_3 \bowtie T_4)$. If the engine discovers an exclusive group consisting of the triple patterns $\{T_2, T_3\}$, this group cannot be integrated into the plan. The plan never produces an intermediate result of the form $T_2 \bowtie T_3$, since these patterns occur in different subtrees. As a result, the exclusive group does not fit into the existing join structure and cannot be pushed as a single subquery. However, if the plan is reordered to $(T_2 \bowtie T_3) \bowtie (T_1 \bowtie T_4)$, this group can be integrated, allowing the sub-query $\{T_2, T_3\}$ to be sent to the appropriate data sources.

3.2.1. Research Avenues

Adaptive query processing offers a clear path forward. It enables low-cost plan switching without discarding intermediate results, preventing plan thrashing caused by the large number of sources in a dataspace. Algorithms such as Eddies [28], SteMs [29] (used in [27]), and STAIRs [30] are viable candidates due to their tuple-level adaptivity.

However, the literature lacks a comprehensive analysis of these adaptive frameworks within LTQP. Specifically, there is little guidance on designing routing strategies, the logic for forwarding tuples between operators, and assessing their overhead costs.

Despite this gap, two works are relevant. The first [27] uses tuple-level routing combined with adaptive URI prioritization. However, they evaluate the performance of the URI prioritization and do not present the performance difference of using tuple-level routing instead of a fixed query plan. As prioritization performs poorly in Solid dataspace [31], these findings lack direct relevance. The second approach [32] uses join restarts to improve query performance. However, if two sources admit incompatible exclusive groups, restarting the join cannot accommodate both.

Given the limitations of these prior works, future research should investigate the use of tuple-level adaptive query processing techniques.

Eddies offer a constrained form of adaptivity. In this model, the join operators are instantiated at the start of the query; the engine only adapts the *order* in which tuples visit these operators. Because these join operators (e.g., Symmetric Hash Joins) accumulate internal state as they process data, the engine is effectively locked into the existing operator instances.

Example 3.2. Consider a query with triple patterns $\{T_1, T_2, T_3, T_4\}$. If an Eddy starts by routing results from $T_1 \bowtie T_2$, the join state is physically stored within that specific operator’s hash tables. If the engine later discovers that $T_2 \bowtie T_3$ is more selective due to newly traversed data, it can change

¹This includes fragments of the URI

the routing sequence, but it cannot easily dissolve the $T_1 \bowtie T_2$ state to form a $T_2 \bowtie T_3$ join without significant re-initialization overhead.

In contrast, **SteMs** (State Modules) decouple state from the join operation entirely. Since SteMs do not hold fixed join states, any tuple can be routed to any SteM in any order. However, this flexibility introduces a *recomputation tax*: if a SteM is probed before its join partners have arrived, the intermediate result is not captured and must be re-derived once more data is discovered via traversal.

Finally, **STAIRs** (Storage, Transformation and Access for Intermediate Results) attempt to bridge this gap through state migration. This allows a query engine to physically move join states between different operator instances. While this maximizes flexibility, state migration is a resource-intensive operation. Determining the optimal threshold for migration, and the number of required migrations to incorporate the diverse precomputed sub-queries discovered during traversal is an open challenge.

3.3. Service Discovery in Hybrid Query Environments

A fundamental challenge in Hybrid Link Traversal Querying is the gap between resource identifiers in Linked Data documents and the aggregated query services that host them. For standard LTQP, the reachability of data is determined by the recursive dereferencing of URIs, adhering to the “follow-your-nose” principle where a URI identifies a resource and provides its description. However, SPARQL endpoints function as aggregators: they contain data about resources but do not necessarily share the resource’s URI namespace, nor are they always explicitly linked from the resource descriptions themselves. Consequently, a query engine may possess a URI for resource A, even though the resource itself is only accessible via a SPARQL endpoint at a different URI. Because of this decoupling, the engine cannot rely on the “follow-your-nose” principle to dereference resource A directly; it must discover the relevant endpoint instead. For example, an engine might encounter the URI `https://alice.example/posts/1` representing a social media post. Dereferencing this URI directly yields no RDF data. Instead, all data regarding Alice’s posts resides exclusively in a SPARQL endpoint located at a different URI, such as `https://alice.example.org/posts/sparql`. Because the resource and endpoint URIs are decoupled, standard link traversal fails to retrieve the data.

3.3.1. Research Avenues

Because standard traversal fails when URIs are decoupled from their access interfaces, query engines must actively discover and map these aggregator interfaces. Several research avenues address this challenge.

Explicit Linkage Mechanisms An avenue for mitigating the service discovery gap is by using explicit links between resources and their hosting endpoints. This approach relies on extending the “follow-your-nose” principle to include service metadata within the resource description itself. Existing methods such as the Vocabulary of Interlinked Datasets (VoID) can embed pointers directly in the RDF data. For instance, a triple taking the form $\langle r, \text{void:sparqlEndpoint}, s \rangle$ allows a traversal engine to immediately identify that the resource r is queryable via the endpoint s . However, this relies on adoption of this approach while, for example, currently only 33% percent of endpoints expose VoID descriptions [33]. Furthermore, generating individual triples for every term may be infeasible. Methods should therefore be investigated to simplify these statements, such as using regex patterns to define what types of resources are constrained in the endpoint.

Service Registries and Indexing. Instead of relying on direct linkage, research can be directed towards the usage of external service registries and catalogs. Unlike linkage-based discovery, this approach treats the mapping between URI namespaces and SPARQL endpoints as a separate knowledge base. Within a dataspace, services such as ESPRESSO [16] can aggregate these connections, effectively bridging the gap between identifiers and query services. However, relying on purely centralized indexes is antithetical to the goals of a decentralized dataspace. Consequently, future directions should

investigate distributed alternatives, such as Peer-to-Peer (P2P) discovery mechanisms, to prevent the reliance on single points of failure.

3.4. Alternative View Deduplication

In decentralized dataspace, a single dataset is often accessible through multiple architectural views. For example, a Solid personal data store (PDS) uses the Linked Data Platform (LDP) to structure data as a hierarchical resource tree. Providers may also expose a SPARQL endpoint over the same PDS to improve efficiency. Ideally, a hybrid FQP-LTQP engine incorporates this endpoint into its query plan. However, because endpoint discovery is a byproduct of traversal, the engine may dereference several hypermedia documents before discovering the metadata pointing towards the endpoint. Due to this, the data ingested from the traversal of the PDS is duplicated when the endpoint is used (Figure 2), leading to duplicated results and wasted computation.

Furthermore, the engine cannot determine what parts of the PDS are aggregated by the endpoint nor if the endpoint aggregates data outside of the PDS. In the specific context of Solid, a client might assume that any (public) URI sharing the Pod’s root namespace is covered by the discovered endpoint. However, in general dataspace environments, this assumption is often unsubstantiated; alternative views may only cover specific subsets of data, such as historical archives versus high-frequency data.

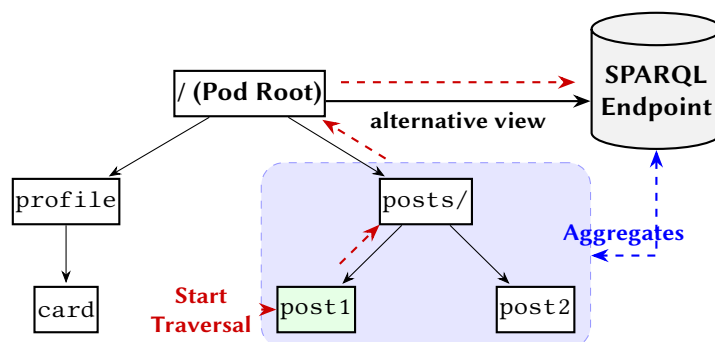


Figure 2: Data coverage overlap and traversal dynamics. Execution starts at post1, traversing up the LDP hierarchy until it discovers the SPARQL endpoint at the root. Because this endpoint acts as an alternative view aggregating the same posts already being traversed, the engine retrieves duplicate data.

3.4.1. Research Avenues

To prevent this duplication, engines must accurately determine the exact data boundaries of overlapping interfaces. Additionally, when an engine ingests duplicate data, it requires an approach that can deduplicate that data internally.

View Overlap Detection To determine an appropriate deduplication strategy, a query engine must first assess the data coverage of the discovered access interfaces. One approach from the literature involves issuing queries to both the local data store and the remote endpoint to calculate *coherence* [11], a metric indicating the extent to which the two sources agree. However, this method is restricted to queryable interfaces and incurs significant overhead, as each coherence check requires additional HTTP requests. An alternative is to rely on server-side metadata that explicitly denotes which underlying resources a given access interface aggregates. Unfortunately, this relies heavily on the widespread adoption of specific metadata vocabularies by data providers, which cannot easily be guaranteed in a decentralized, autonomous dataspace.

Client-Side Deduplication Regardless of whether the data coverage of discovered access interfaces is known, the query engine must perform client-side deduplication. This requires tracking the *provenance* (i.e., the source) of intermediate results to filter out duplicates.

We consider two scenarios based on this coverage knowledge. First, without coverage guarantees, the engine relies on a naive approach. It maintains a “seen” set of all previously produced intermediate results for each triple pattern, discarding any new matches that already exist in the set. Retaining this complete history is highly memory-intensive and scales poorly. While probabilistic data structures like Bloom filters can reduce this memory footprint, their inherent false-positive rates risk accidentally discarding valid results.

Second, if the engine knows which sources an alternative access interface covers, it can optimize the lifecycle of the “seen” sets. Once the engine guarantees that all data capable of producing duplicate results has been processed, it can safely discard the associated sets to free memory and prevent sources covered by the alternative interface from being dereferenced. However, reliably detecting this execution point remains an open research problem.

4. Conclusion

Hybrid querying can reduce query latency and network overhead by combining dynamic link traversal with the efficiency of expressive interfaces. However, realizing a fully functional Hybrid FQP-LTQP engine requires overcoming significant technical hurdles. Engines must establish exclusive groups without prior knowledge, adapt query plans dynamically to integrate discovered sub-queries, implement robust service discovery to link resource identifiers with aggregating endpoints, and manage alternative views to prevent result duplication. For approaches that rely on metadata to guide traversal and query planning, standardization efforts within dataspace are required to facilitate them. Ultimately, to scale querying across massive and heterogeneous dataspace, engines must stop relying solely on client-side joins of single triple patterns. Instead, they must dynamically delegate complex sub-queries to expressive interfaces to reduce network transfer and leverage server-side computation.

Acknowledgements This research was supported by SolidLab Vlaanderen (Flemish Government, EWI and RRF project VV023/10). Ruben Taelman is a postdoctoral researcher at the Research Foundation – Flanders (FWO).

Declaration on Generative AI During the writing of this paper, the author(s) used Gemini in order to: Sentence Polishing, Rephrasing, and Text Creation. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication’s content.

References

- [1] M. Schmidt, O. Görlitz, P. Haase, G. Ladwig, A. Schwarte, T. Tran, Fedbench: A benchmark suite for federated semantic data query processing, in: ISWC 2011, Springer, 2011, pp. 585–600.
- [2] M.-H. Dang, J. Aimonier-Davat, P. Molli, O. Hartig, H. Skaf-Molli, Y. Le Crom, Fedshop: a benchmark for testing the scalability of sparql federation engines, in: International Semantic Web Conference, Springer, 2023, pp. 285–301.
- [3] S. Cheng, O. Hartig, Towards query processing over heterogeneous federations of rdf data sources, in: European Semantic Web Conference, Springer, 2022, pp. 57–62.
- [4] L. Heling, M. Acosta, Federated sparql query processing over heterogeneous linked data fragments, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 1047–1057.
- [5] R. Verborgh, M. Vander Sande, O. Hartig, J. Van Herwegen, L. De Vocht, B. De Meester, G. Haesendonck, P. Colpaert, Triple pattern fragments: a low-cost knowledge graph interface for the web, *Journal of Web Semantics* 37 (2016) 184–206.
- [6] A. Azzam, C. Aebeloe, G. Montoya, I. Keles, A. Polleres, K. Hose, Wisekg: Balanced access to web knowledge graphs, in: Proceedings of the Web Conference 2021, 2021, pp. 1422–1434.
- [7] B. Esteves, H. J. Pandit, V. Rodríguez-Doncel, Odl profile for expressing consent through granular access control policies in solid, in: 2021 IEEE EuroS&PW, IEEE, 2021, pp. 298–306.

- [8] A. Padia, T. Finin, A. Joshi, et al., Attribute-based fine grained access control for triple stores, in: 3rd Society, Privacy and the Semantic Web-Policy and Technology workshop, 2015.
- [9] E. Curry, Dataspaces: fundamentals, principles, and techniques, in: Real-time Linked Dataspaces: Enabling Data Ecosystems for Intelligent Systems, Springer, 2019, pp. 45–62.
- [10] J. Van Herwegen, R. Verborgh, Granular access to policy-governed linked data via partial server-side query, in: European Semantic Web Conference, Springer, 2024, pp. 331–335.
- [11] J. Umbrich, A hybrid framework for querying linked data dynamically (2012).
- [12] S. J. Lynden, I. Kojima, A. Matono, A. Nakamura, M. Yui, A hybrid approach to linked data query processing with time constraints., LDOW 996 (2013) 1–10.
- [13] O. Hartig, K. Hose, J. F. Sequeda, Linked data management., 2019.
- [14] G. Ladwig, T. Tran, Linked data query processing strategies, in: International Semantic Web Conference, Springer, 2010, pp. 453–469.
- [15] G. Ladwig, T. Tran, Sihjoin: Querying remote and local linked data, in: ESWC, Springer, 2011, pp. 139–153.
- [16] M. Ragab, Y. Savateev, R. Moosaei, T. Tiropanis, A. Poulouvassilis, A. Chapman, G. Roussos, Espresso: a framework for empowering search on decentralized web, in: International Conference on Web Information Systems Engineering, Springer, 2023, pp. 360–375.
- [17] S. Speicher, J. Arwe, A. Malhotra, Linked data platform 1.0, 2015. URL: <https://www.w3.org/TR/ldp/>, w3C Recommendation.
- [18] R. Taelman, J. Van Herwegen, M. Vander Sande, R. Verborgh, Comunica: a modular sparql query engine for the web, in: International Semantic Web Conference, Springer, 2018, pp. 239–255.
- [19] R. Taelman, R. Verborgh, Link traversal query processing over decentralized environments with structural assumptions, in: International Semantic Web Conference, Springer, 2023, pp. 3–22.
- [20] A. Schwarte, P. Haase, K. Hose, R. Schenkel, M. Schmidt, Fedx: Optimization techniques for federated query processing on linked data, in: ISWC 2011, Springer, 2011, pp. 601–616.
- [21] J. Aimonier-Davat, B. Nédelec, M.-H. Dang, P. Molli, H. Skaf-Molli, Fedup: querying large-scale federations of sparql endpoints, in: Proceedings of the ACM Web Conference 2024, 2024, pp. 2315–2324.
- [22] M. Saleem, A.-C. Ngonga Ngomo, Hibiscus: Hypergraph-based source selection for sparql endpoint federation, in: European semantic web conference, Springer, 2014, pp. 176–191.
- [23] O. Hartig, J.-C. Freytag, Foundations of traversal based query execution over linked data, in: Proceedings of the 23rd ACM conference on Hypertext and social media, 2012, pp. 43–52.
- [24] O. Hartig, SQUIN: a traversal based query execution system for the web of linked data, in: Proc. SIGMOD, 2013, pp. 1081–1084.
- [25] B. Bogaerts, B. Ketsman, Y. Zeboudj, H. Aamer, R. Taelman, R. Verborgh, Distributed subweb specifications for traversing the web, Theory and Practice of Logic Programming 24 (2024) 394–420.
- [26] R. Verborgh, R. Taelman, Guided link-traversal-based query processing, arXiv preprint arXiv:2005.02239 (2020).
- [27] O. Hartig, M. T. Özsu, Walking without a map: Ranking-based traversal for querying linked data, in: International Semantic Web Conference, Springer, 2016, pp. 305–324.
- [28] R. Avnur, J. M. Hellerstein, Eddies: Continuously adaptive query processing, in: Proceedings of the 2000 ACM SIGMOD international conference on Management of data, 2000, pp. 261–272.
- [29] V. Raman, A. Deshpande, J. M. Hellerstein, Using state modules for adaptive query processing, in: Proceedings 19th International Conference on Data Engineering, IEEE, 2003, pp. 353–364.
- [30] A. Deshpande, J. M. Hellerstein, Lifting the burden of history from adaptive query processing, in: Proc. 30th VLDB Conf., 2004.
- [31] R. Eschauzier, R. Taelman, R. Verborgh, Revisiting link prioritization for efficient traversal in structured decentralized environments, in: ISWC, Springer, 2025, pp. 575–593.
- [32] J. Hanski, S. Van Braeckel, R. Taelman, R. Verborgh, Link traversal over decentralised environments using restart-based query planning, in: ICWE, Springer, 2025, pp. 303–311.
- [33] C. Buil-Aranda, A. Hogan, J. Umbrich, P.-Y. Vandenbussche, Sparql web-querying infrastructure: ready for action?, in: International Semantic Web Conference, Springer, 2013, pp. 277–293.